

Raport Projektowy: System Bazy Danych dla Sieci Bibliotek

Bartosz Stasiak
Kerem Özdoğan

17 grudnia 2025

Spis treści

1	Wstęp i cel projektu	3
2	Schemat bazy danych	3
3	Warunki integralnościowe	4
3.1	Klucze obce i główne	4
3.2	Walidacja danych (CHECK)	4
4	Procedury składowane i Wyzwalacze (Triggers)	5
4.1	Automatyczna zmiana statusów	5
4.2	Zaawansowana walidacja dat (Cross-table)	5
5	Raportowanie (Widoki)	5
6	Zasady bezpieczeństwa	6
7	Testy funkcjonalne	6
7.1	Test 1: Automatyzacja statusów	6
7.2	Test 2: Blokada błędnych dat	6
8	Podsumowanie	7

1 Wstęp i cel projektu

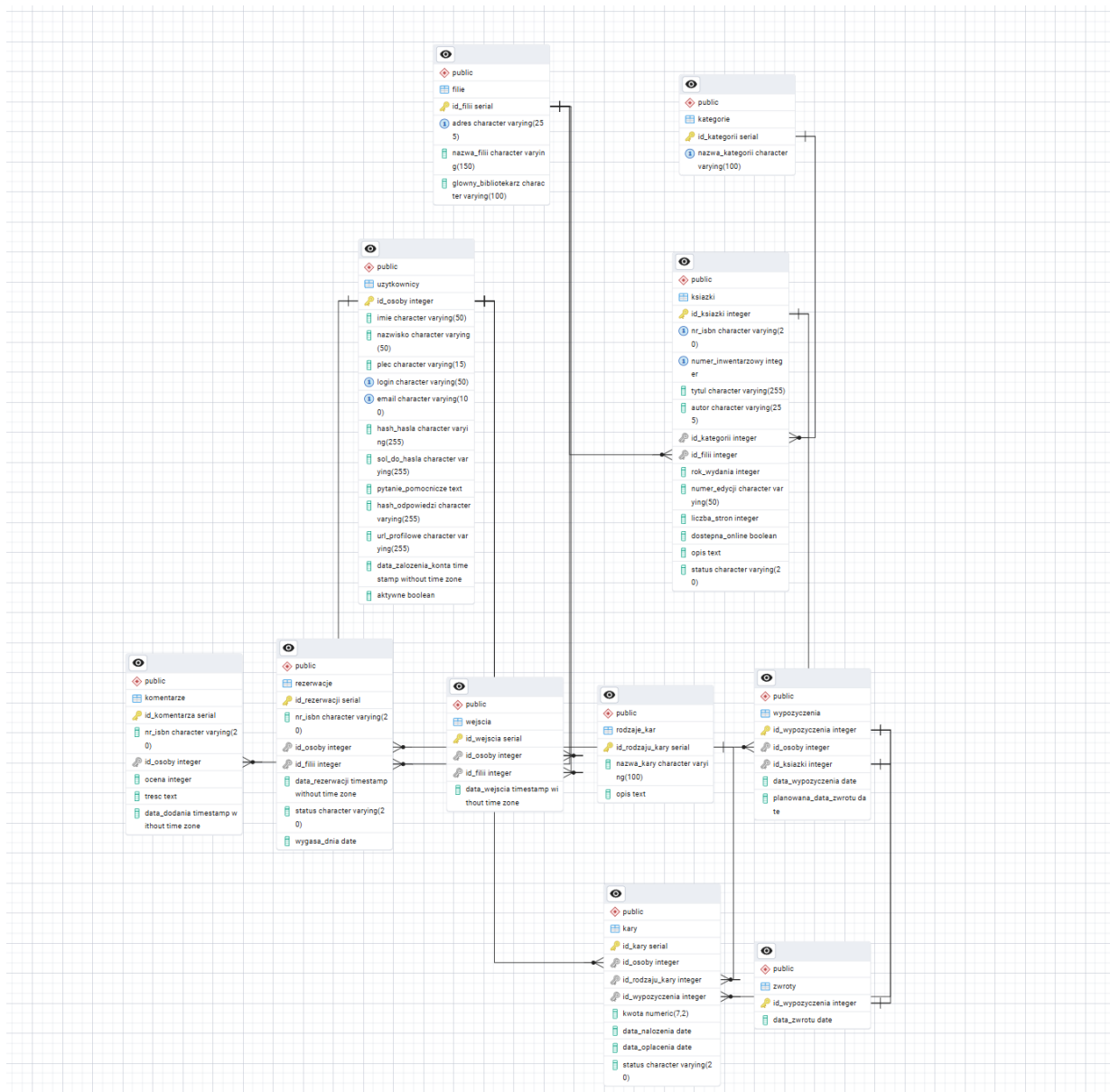
Celem projektu było zaprojektowanie i implementacja relacyjnej bazy danych obsługującej system zarządzania siecią bibliotek miejskich. System umożliwia ewidencję księgozbioru, obsługę czytelników, rejestrację wypożyczeń i zwrotów oraz naliczanie kar.

Główny nacisk położono na **automatyzację procesów** (wyzwalacze) oraz **bezpieczeństwo i integralność danych**. Baza została zaimplementowana w systemie **PostgreSQL**.

2 Schemat bazy danych

Baza danych została znormalizowana do Trzeciej Postaci Normalnej (3NF). Składa się z 12 powiązanych tabel, podzielonych na moduły logiczne:

- **Zasoby:** książki, kategorie, filie.
- **Użytkownicy:** użytkownicy (dane osobowe, loginy).
- **Operacje:** wypożyczenia, zwroty, rezerwacje, wejścia.
- **Kontrola:** kary, rodzaje_kar, komentarze.



Rysunek 1: Diagram ERD (Entity Relationship Diagram) systemu.

3 Warunki integralnościowe

W celu zapewnienia spójności danych zastosowano szereg ograniczeń (Constraints).

3.1 Klucze obce i główne

Każda tabela posiada klucz główny (PRIMARY KEY). Relacje między tabelami są zabezpieczone kluczami obcymi (FOREIGN KEY) z opcją ON DELETE RESTRICT lub CASCADE w zależności od logiki biznesowej (np. nie można usunąć czytelnika, który posiada nieoddane książki).

3.2 Walidacja danych (CHECK)

Zastosowano ograniczenia logiczne dla wprowadzanych danych:

```
-- Ocena książki musi być w przedziale 1-10
CONSTRAINT komentarze_ocena_check CHECK ((ocena >= 1) AND (ocena <= 10))
```

```

-- Płeć ograniczona do słownika
CONSTRAINT osoby_plec_check CHECK (((plec)::text = ANY
  ((ARRAY['MEZCZYZNA'::character varying, 'KOBIETA'::character varying,
    'NIEOKRESLONY'::character varying])::text [])))

-- Data planowanego zwrotu nie może być wcześniejsza niż data wypożyczenia
ALTER TABLE wypozyczenia
ADD CONSTRAINT check_planowana_data
CHECK (planowana_data_zwrotu >= data_wypozyczenia);

```

Listing 1: Przykłady ograniczeń CHECK

4 Procedury składowane i Wyzwalacze (Triggers)

Automatyzacja jest kluczowym elementem systemu, eliminującym błędy ludzkie (np. zapomnienie o zmianie statusu książki).

4.1 Automatyczna zmiana statusów

Zaimplementowano dwa wyzwalacze dbające o aktualność pola **status** w tabeli **ksiazki**.

- **Wypożyczenie:** Trigger zmienia status na 'wypożyczony' po dodaniu rekordu do tabeli wypozyczenia.
- **Zwrot:** Trigger przywraca status 'dostępny' po dodaniu rekordu do tabeli zwroty.

4.2 Zaawansowana walidacja dat (Cross-table)

Ponieważ standardowy CHECK nie pozwala porównywać danych między dwiema tabelami, stworzono funkcję w języku PL/pgSQL, która blokuje próbę zwrotu z datą wcześniejszą niż data wypożyczenia.

```

CREATE OR REPLACE FUNCTION sprawdz_date_zwrotu() RETURNS trigger AS $$
DECLARE
    v_data_wypozyczenia DATE;
BEGIN
    SELECT data_wypozyczenia INTO v_data_wypozyczenia
    FROM wypozyczenia
    WHERE id_wypozyczenia = NEW.id_wypozyczenia;

    IF NEW.data_zwrotu < v_data_wypozyczenia THEN
        RAISE EXCEPTION 'Błąd: Data zwrotu nie może być wcześniejsza niż
            data wypożyczenia';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

Listing 2: Funkcja walidująca datę zwrotu

5 Raportowanie (Widoki)

Dla ułatwienia pracy bibliotekarzy stworzono widoki (Views), które agregują dane.

```
CREATE OR REPLACE VIEW view_dluznicy AS
SELECT
    u.imie, u.nazwisko, u.email, k.tytul,
    (CURRENT_DATE - w.planowana_data_zwrotu) as dni_spoznienia
FROM wypozyczenia w
JOIN uzytkownicy u ON w.id_osoby = u.id_osoby
JOIN ksiazki k ON w.id_ksiazki = k.id_ksiazki
LEFT JOIN zwroty z ON w.id_wypozyczenia = z.id_wypozyczenia
WHERE z.data_zwrotu IS NULL AND w.planowana_data_zwrotu < CURRENT_DATE;
```

Listing 3: Widok listy dłużników

6 Zasady bezpieczeństwa

Zastosowano zasadę najmniejszych przywilejów (Least Privilege Principle), tworząc role z różnym poziomem dostępu.

```
-- Rola dla pracownika (pełny dostęp do danych)
CREATE ROLE bibliotekarz WITH LOGIN PASSWORD '***';
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO bibliotekarz;

-- Rola dla gościa/kiosku informacyjnego (tylko odczyt katalogu)
CREATE ROLE gosc_biblioteki WITH LOGIN PASSWORD '***';
GRANT SELECT ON ksiazki, kategorie, filie TO gosc_biblioteki;

-- Blokada dostępu do danych osobowych dla gościa
REVOKE ALL ON uzytkownicy FROM gosc_biblioteki;
```

Listing 4: Konfiguracja ról i uprawnień

7 Testy funkcjonalne

Przeprowadzono testy weryfikujące działanie logiki biznesowej.

7.1 Test 1: Automatyzacja statusów

Cel: Sprawdzenie, czy wypożyczenie zmienia status książki.

```
-- Krok 1: Wstawienie wypożyczenia
INSERT INTO wypozyczenia (id_osoby, id_ksiazki, data_wypozyczenia,
    planowana_data_zwrotu)
VALUES (1, 1, CURRENT_DATE, CURRENT_DATE + 30);

-- Krok 2: Weryfikacja
SELECT status FROM ksiazki WHERE id_ksiazki = 1;
-- Wynik: 'wypożyczony' (TEST ZALICZONY)
```

7.2 Test 2: Blokada błędnych dat

Cel: Próba oddania książki z datą z przeszłości (sprzed wypożyczenia).

```
-- Próba wstawienia błędnego zwrotu
INSERT INTO zwroty (id_wypozyczenia, data_zwrotu)
VALUES (100, '2020-01-01');
```

```
-- Wynik: BŁĄD SQL: "Data zwrotu nie może być wcześniejsza niż data  
wypożyczenia"  
-- (TEST ZALICZONY - Trigger zadziałał)
```

8 Podsumowanie

Projekt spełnia wszystkie założenia systemu bazodanowego dla biblioteki. Struktura jest znormalizowana, dane są chronione przed błędami logicznymi, a dostęp jest zabezpieczony systemem ról. System jest gotowy do wdrożenia produkcyjnego.